

DMM6500 behaviour the manuals do not state

Keithley publishes an example repository at `github.com/tektronix/keithley`. These notes come from reading it — `Instrument_Examples/`, `Application_Specific/`, `TTI_Apps/` and `Drivers/`, 217 TSP files and their host-side callers — and checking every call it makes against the DMM6500 reference manuals (`DMM6500-901-01A` April 2018 and `DMM6500-901-01B` September 2019), the user manual, and the display and TSP-app documents that ship beside the examples.

A fact earns a place here only if an example demonstrates or depends on it **and** the reference manual is silent, lists the call without explaining it, or contradicts it. Where a vendor display document is the contradicting source rather than the reference manual, this file names it.

The instrument is the arbiter, and on the trigger model it has to be. Keithley's own application code never reads a `trigger.` setting back — `GAS/MS01_SetupAndRun.tsp` performs about twenty writes under `trigger.` and `tsplink.` and checks none of them, no `getblocklist()`, no attribute read, no event-log drain. A refused write under `trigger.` files an event rather than failing, so an example shows what Keithley intended, not what the firmware took. That is why every probe below reads a value back.

Every claim here has been to the instrument, so nothing is left standing as a hypothesis. What is not measured is refuted, or out of reach on this bench for a stated physical reason rather than for want of trying. A few claims are measured in part and say so in their own sentence.

There is deliberately no tally in this paragraph. Two writers re-derived one by hand within an hour and it went adrift twice, once by a claim and once by two statuses. Count the status tokens if you want the figures; a number nobody can recompute from the file is worse than no number.

Status	Meaning
MEASURED	Confirmed on this DMM6500, firmware 1.7.17a.
REFUTED	The instrument disagreed. The claim stays, with what actually happens.
UNTESTABLE HERE	Out of reach on this bench, with the reason in the same sentence — a human at the front panel, a second instrument, an accessory card, a power cycle. Never "hard".

Two claims were **cut** rather than given a status, because the manual turns out to state both in plain English and they failed this file's own bar: digitize having no autorange, stated at `DMM6500-901-01B:7730` and again at 8402 with `dmm.digitize.range`'s own Details calling the range "fixed"; and `buffer.save()` taking the `SAVE_*` constants, whose own Example section uses them twice. Both were true, and neither was undocumented.

Under each claim is its bench probe, indented and in backticks. Four rules hold across them. A probe that changes instrument state carries its own restore step. No probe rests on a `pcall` verdict — each one reads the attribute back, reads `getblocklist()`, or drains and prints `eventlog.next(eventlog.SEV_ALL)`; on the file API a `pcall` verdict is a constant, because no `file.*` call raises, and under `trigger.` a refused `setblock` and a refused `buffer.make` both return success. Every probe prints one **tagged** value, because the instrument volunteers event lines on the control socket whenever `localnode.showevents` is set, and an untagged read then returns an event where the reply should be and leaves every later read one behind — measured, eight replies adrift. So the probes run with `showevents = 0` and restore it afterwards. A model is polled from **inside** the instrument in every claim but one, because a host query aborts an initiated model; the exception is the claim where that arriving command is itself the experiment. Where a claim needs a front-panel key, the press goes through the instrument's own web panel rather than a finger, and the claim says so.

TSP scripting, apps and the host interface

No DMM6500 manual mentions the display object API, and \$Requires does not guarantee a symbol

MEASURED. `display.create`, `display.setkeyenable`, `OBJ_TEXT` and `display.EVENT_*` each occur zero times in both reference manuals and zero times in the user manual. The API is specified only in `DMM6500DisplayDoc.md`, which Keithley distributes separately, and in `Display API Command Reference.md`, which ships at `TTI_Apps/TTI_Display_API/`. Version does not stand in for a feature test: `TTI_Apps/Pong.tspa` declares `-- $Requires: v1.7.0` and still reads `if display.setkeyenable then` at :205 and `if display.EVENT_ENDAPP then` at :222. On 1.7.17a every symbol Pong guards is present — `display.create` and `display.setkeyenable` are functions, `display.EVENT_ENDAPP`, `display.EVENT_PRESS` and `display.OBJ_TEXT` all non-nil — so the guard costs nothing here and the feature test remains the only way to ask.

- probe:

```
print("REQ|ver=..toString(localnode.version)..|create=..type(display.create)..|ske=..type(display.setkeyenable)..|endapp=..toString(display.EVENT_ENDAPP)..|ep=..toString(display.EVENT_PRESS)..|objtext=..toString(display.OBJ_TEXT)|" — the manual half stays a property of the manuals, settled by grep.
```

Ten numbered fonts exist beside the four named ones, and display.FONT_11 is nil

MEASURED. Confirmed on firmware 1.7.17a. `FONT_SMALL`, `FONT_MEDIUM`, `FONT_LARGE` and `FONT_HUGE` exist too. Neither reference manual mentions any font constant — zero hits for `FONT_` in both.

- probe:

```
do local o="" for i=1,11 do o=o..i.."="..type(display["FONT_"..i]).." " end print(o, type(display.FONT_SMALL), type(display.FONT_MEDIUM), type(display.FONT_LARGE), type(display.FONT_HUGE)) end
```

display.create takes a numbered font although its own doc note lists only four

MEASURED. `Display API Command Reference.md:1446-1459` lists all fourteen under `display.setfont()`, marking `FONT_1 - FONT_10` (v1.7+); its `display.create()` note for `OBJ_TEXT` at :1291-1294 lists four, and `DMM6500DisplayDoc.md:251` offers only "Large, Medium, or Small". Three files pass a numbered font straight to `display.create`: `TTI_Apps/CalculatorApp.tspa:198` and

TTI_Apps/DAQ6510_MultiFuncCtrl.tsipa:493 use FONT_7 , and Series_2400_Graphical/SourceConstantPower24xx.tsp:119-129 uses FONT_6 and FONT_4 . The tree never calls display.setFont at all, and uses only three of the ten numbered sizes.

display.create accepts all ten: ten non-nil ids under one screen. display.setFont is a function too, and it takes a numbered font and a named one on a live object with nothing filed — but it does not erase what was there, so the old glyphs stay on the glass under the new ones until something repaints.

The ten are not a size ladder. Rendered as "nWg" and measured off the grab, heights run 15, 17, 20, 21, 26, 29, 32, 105, 21, 121 px. FONT_8 is 105 px, the same as FONT_HUGE ; FONT_9 drops back to 21 px and is the only *italic* face in the set; FONT_10 at 121 px clips a four-character string. So a number chosen for size alone can silently change the typeface or overflow its object.

- probe: do KTF={} KTS=display.create(display.ROOT,display.OBJ_SCREEN,"ktfonts") local x,y for i=1,10 do if i<=5 then x=15 y=15+(i-1)*90 else x=410 y=15+(i-6)*90 end KTF[i]=display.create(KTS,display.OBJ_TEXT,x,y,i.."Wg",0xFFFFFF,display["FONT_"..i]) end print("FONTS|scr="..tostring(KTS).."|f1="..tostring(KTF[1])) end then pcall(display.changescreen,KTS) and grab the screen. A 90 px pitch runs 7 into 8 and 9 into 10; re-render those five at 165 px to measure them. Tear down every id and the screen; all are in named globals, so none can be lost.

display.setkeyenable turns off a front-panel key, including TRIGGER

MEASURED. TTI_Apps/Pong.tsipa:206-209 disables display.KEY_FUNCTION , display.KEY_HELP , display.KEY_TRIGGER and display.KEY_QUICKSET_APPS . No reference manual or display document mentions setkeyenable or any display.KEY_ constant. **The TRIGGER key is an escape route only while nothing has disabled it**, and display.OFF really does stop it posting trigger.EVENT_DISPLAY .

A model parked in BLOCK_WAIT , trigger.EVENT_DISPLAY with a one-reading measure block behind it, four arms, pressing the key the same way each time:

arm	result
no press at all	n = 0 , STATE_WAITING at block 1
key enabled, pressed	n = 1 , STATE_IDLE
key disabled, pressed	n = 0 , STATE_WAITING
key enabled again, pressed	n = 1 , STATE_IDLE

Exactly those four display.KEY_ constants exist — KEY_HOME , KEY_MENU , KEY_EXIT and KEY_ENTER are nil, so only Pong's set can be touched. **There is no getter:** display.getkeyenable is nil and setkeyenable returns nothing and files nothing, so a disabled key cannot be read back — arming and pressing is the only way to ask. Nothing in this app calls setkeyenable .

- probe: the press is injected through the instrument's own web front panel, not a finger – POST /ajax_proc with function=7 (BUTTON_PRESS_FUNCTION), button=8 (Trigger), session=<id>, then function=8 to release, read out of the instrument's /script/ajax_inc.js. The key-enable filter applies to that path, which is what makes the crossover evidence. Poll from INSIDE: do defbuffer1.clear() dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=1000 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_WAIT,trigger.EVENT_DISPLAY) trigger.model.setblock(2,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,1) trigger.model.initiate() delay(4) local a,b,c=trigger.model.state() print("KP|n="..tostring(defbuffer1.n).."|st="..tostring(a)) trigger.model.abort() end , with the press landing 1.5 s in. A HOST readback after the press measures nothing — the query aborts the model, see below.
- Restore every key with display.ON on the way out, or an interrupted run leaves the front panel's escape dead.

The display doc's two app-header rules are both wrong

UNTESTABLE HERE — the Apps menu is the only thing that parses these tags, so settling it needs a .tsipa installed from the USB key and a human reading the menu. The evidence stands without a run: DMM6500DisplayDoc.md:684,703 states two constraints, and the shipped apps break both unanimously.

Rule in the doc	What the apps do
appName in loadimage "must match the name of the app used for the \$Title: tag"	loadimage 's second argument is the loadscript identifier. TTI_Apps/CalculatorApp.tsipa opens loadscript CalcApp , titles itself TSP Calculator , and closes loadimage calc_icon CalcApp . All eight icon-bearing apps do this.
A title "must not have spaces"	Seven of nine shipped apps use them, including -- \$Title: Res Tolerance .

- no probe: a loadimage block INSTALLS an icon, and nothing in either manual or the display document removes one. On an instrument with a live app loaded that is an unbounded change to make for a readback, so it was left alone rather than run.
- no probe: which name the Apps menu binds the icon to, and whether a spaced title installs, need a USB install and a human reading the glass

A display escape code needs a trailing space only before a digit

MEASURED. Lua reads up to three decimal digits after a backslash, so only a following digit is ambiguous. TTI_Apps/Probe_Hold.tsipa:115 writes "-\21-\21.\21-\21-\21-\21" and TTI_Apps/Resistance_Tolerance_Meter.tsipa:148 writes string.format("%.5f\21\18", reading) . DMM6500DisplayDoc.md:628 says the codes "must have a space after them". On the instrument "\21\18" is **two** bytes, 21 and 18; "\218" is

one byte, 218; `"\21 "` is two. So the doc's rule is right about the ambiguity and wrong to make it general — the space is needed only where a digit follows, and `Resistance_Tolerance_Meter` 's unspaced pair is correct as written.

- `probe:`
`print("ESC|len2="..string.len("\21\18").."|b1="..string.byte("\21\18",1).."|b2="..string.byte("\21\18",2).."|len1="..string.len("\218").."|b218="..string.byte("\218",1).."|lensp="..string.len("\21 "))`

eventlog.suppress(N) keeps event N's modal dialog off the panel, and the log entry stays

MEASURED. `Application_Specific/Battery_Test/DC-IR/Battery_DCIR.tspa:614-617` calls it four times — 5076, 2732, 2731, 5078 — straight after `eventlog.clear()`. "suppress" has zero occurrences in both reference manuals and the user manual; the documented set is `clear`, `csv`, `getcount`, `next`, `post` and `save`. That app targets an SMU, and the DMM6500 takes it.

What it suppresses is the **dialog, not the entry**. The vendor repro at `vendor/repro-02-4915.tsp` — a `LoopUntilEvent` model over a 20,000-reading `FILL_ONCE` buffer at 1 MS/s, armed 3 s on the TRIGGER key — files the same twelve entries either way: 2731, ten 4915, 2728. With `eventlog.suppress(4915)` called first, the panel stays clean; from a fresh socket with no such call, the same statement puts **Error 4915** over the app's own screen. One call covers every capture in that session — two provocations, no dialog.

It is scoped to the socket session, and there is no inverse. `eventlog.unsuppress` is nil, as are `eventlog.count` and `eventlog.csv`. `suppress(4915,0)`, `suppress(4915,1)` and `suppress(0)` all return without raising and change nothing; a bare `suppress()` files **-109 Missing parameter** and raises its own dialog. A power cycle is not needed — a new connection has 4915 back.

4915 is only the first dialog an armed capture raises. Suppressing it reveals **Warning 2728** *"Trigger model path #1 has been aborted"*, filed by the model's own `abort()`. Suppressing 4915 and 2728 together leaves the panel clean through the whole provocation.

- `probe:` `do local a=pcall(eventlog.suppress,4915) local b=pcall(eventlog.suppress,2728) eventlog.clear() KB=buffer.make(20000,buffer.STYLE_STANDARD) KB.fillmode=buffer.FILL_ONCE dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.count=20000 dmm.digitize.samplerate=1000000 trigger.model.load("LoopUntilEvent",trigger.EVENT_DISPLAY,5,trigger.CLEAR_ENTER,0,KB) trigger.model.initiate() delay(3) trigger.model.abort() delay(0.5) print("BOTH|a="..tostring(a).."|b="..tostring(b).."|ev="..tostring(eventlog.getcount())) end` — then grab the screen, because the panel is the verdict and the count is the same either way. The control is the same statement from a fresh socket with both pcalls removed.

A modal event dialog is cleared by display.changescreen()

MEASURED. `DMM6500DisplayDoc.md` gives `changescreen` no effect on dialogs, and `vendor/02-event-4915-unmutable.md` records that the box "stays until somebody presses a button" — which made an unattended instrument look unrecoverable. It is not: a bare `eventlog.suppress()` raises an **Error -109** box, and one `display.changescreen(scr)` onto the screen already in front takes it away. The log entry survives the dismissal — `eventlog.next` still returns -109 afterwards — so nothing is lost by clearing the glass.

- `probe:` `do pcall(eventlog.suppress) delay(1) print("RAISE|ev="..tostring(eventlog.getcount())) end` then `do local ok=pcall(display.changescreen,sdec.ui_scr) delay(0.5) print("CLEARDLG|ok="..tostring(ok)) end`, with a screen grab after each

The instrument runs Lua 5.0.2, but its lexer takes | and 0b — two of these three idioms are fine

REFUTED in part. `_VERSION` reads `Lua 5.0.2`. Of the three idioms this entry called broken, only two are.

Idiom	What happens
<code>math.fmod</code>	Absent, as claimed. <code>math.fmod</code> is nil and <code>math.mod</code> is a function. Twelve files use <code>math.mod</code> , including <code>TTI_Apps/DI0ControlFull.tspa:29</code> and three DMM6500 streaming helpers.
<code>eventlog.SEV_WARN eventlog.SEV_ERROR</code> , and <code>digio.writeport(0b110101)</code> under "You can write binary, decimal or hexadecimal values"	Both work. <code>1 2</code> evaluates to 3 and <code>0b110101</code> to 53 — Keithley's build extends the 5.0.2 lexer with a bitwise-or operator and a binary literal, so the manual is right and this entry was wrong. <code>bit.bitor(1,2)</code> is 3 and <code>bit.bitand(6,3)</code> is 2; <code>bit.bor</code> is nil. <code>eventlog.SEV_WARN</code> is 2. <code>TTI_Apps/Pong.tspa:518</code> keeps its mask line commented out for no reason that holds here.
<code>present_state == (trigger.STATE_WAITING or trigger.STATE_RUNNING)</code> , at <code>Amp-Hour_Measurement/Amp-Hours.tsp:80</code>	Dead arm, as claimed. <code>STATE_WAITING</code> or <code>STATE_RUNNING</code> yields <code>STATE_WAITING</code> , and <code>STATE_RUNNING == (STATE_WAITING or STATE_RUNNING)</code> is false.

- `probe:` compile it and then CALL it, because compiling is not evaluating — `do local f=loadstring("return 1|2") local g=loadstring("return 0b110101") local a,b,c,d if f~=nil then a,b=pcall(f) end if g~=nil then c,d=pcall(g) end print("LUA2|orval="..tostring(b).."|bval="..tostring(d).."|bitor="..type(bit.bitor)) end` — `loadstring` returning a function was the first reading, and on its own it looks like a successful compile of something that cannot run.
- `probe:` `do local w,r=trigger.STATE_WAITING,trigger.STATE_RUNNING print("ORTRAP|wor="..tostring(w or r).."|eq="..tostring(r==(w or r))) end`

The base library is Lua 5.0.2's, not the five functions the manual lists

MEASURED. The manual's "Base library functions" table holds `collectgarbage`, `gcinfo`, `tonumber`, `tostring` and `type`, and names `bit`, `io` and `opc` only among words you cannot use as variables. All twelve asked for are functions: `pcall`, `loadstring`, `unpack`, `pairs`, `ipairs`, `next`, `error`, `assert`, `gcinfo`, `dofile`, `opc` and `setmetatable`. `Application_Specific/Electrochemistry/EC_Framework.tsp` is built on `loadstring`, synthesising display code at run time in about twenty places (`:102`, `:148`, `:316`), and uses the tree's only `pcall` at `:402` to guard a compile. `unpack` appears in 21 files.

- probe:


```
print("BASE|pcall=..type(pcall)..|loadstring=..type(loadstring)..|unpack=..type(unpack)..|pairs=..type(pairs)..|ipairs=..type(ipairs)..|next=..type(next)..|error=..type(error)..|assert=..type(assert)..|gcinfo=..type(gcinfo)..|dofile=..type(dofile)..|opc=..type(opc)..|setmeta=..type(setmetatable))
```

Lua's io library is present, and it reports a failure that file.open cannot

MEASURED. `io` is a table with `open`, `close`, `read` and `write`, and a read-open of an absent file returns `nil` plus `"/usb1/KTN0SUCH.XYZ: No such file or directory"` — a named reason, where `file.open` gives no signal at all and files nothing. `io` handles are objects — `f:read()`, `f:write()`, `f:flush()` — not the integer descriptors `file.*` returns. `EC_Framework.tsp:389` reads `local f = io.open('/usb1/'..filename, 'r')` then `f:read(); :442` and `:481` open `'w+'` and test if `f == nil` then; `Application_Specific/Ford EMC-CS-2009.1 specification/AWGforEMC.tsp:409` takes both returns as `file, err = io.open(filepath, "r")`. `io` appears in the reference manual only as a reserved word, and its 13 apparent `io.write` hits are all inside `digio.writeport`.

- probe: `do local f,e=io.open("/usb1/KTN0SUCH.XYZ","r")`
`print("IOOPEN|f=..toString(f)..|type=..type(f)..|err=..toString(e)) if f then pcall(io.close,f) end end` — a read-open of an absent file creates nothing; the close is in a `pcall` because a record left open needs a power cycle.

There is no 8.3 limit on /usb1 — a computed long name is written whole

REFUTED. By hand the vendor writes to the limit — `"/usb1/TestData.csv"`, `"/usb1/DuraQAAA.csv"`, `"/usb1/KI_setup.set"`. By program it writes past it: `Instrument_Examples/DMM6500/Save_Measurement.tsp:16-17` builds `file.open("/usb1/"..description.."csv", ...)` out of `display.input.string`, and `GAS/MS02_ExportData.tsp:22` uses `os.date("/usb1/data_%Y-%m-%d_%H-%M-%S.csv")`. The manual states no name limit **and there is none to state**: `file.open("/usb1/KTLONGNAME.CSV", file.MODE_WRITE)` returns a handle with zero events, and the file re-opens for read under **that full name** while the truncated `KTLONGNAM.CSV` does not exist. So the vendor's computed paths are safe and the hand-written 8.3 names are a habit, not a requirement.

- probe: a write handle is not evidence — the file API never raises, so the name could have been truncated under it. The read-open is the proof: `do local a=file.open("/usb1/KTLONGNAME.CSV",file.MODE_READ) local b=file.open("/usb1/KTLONGNAM.CSV",file.MODE_READ) print("EXISTS|long=..toString(a)..|trunc=..toString(b)) if a~=nil then pcall(file.close,a) end if b~=nil then pcall(file.close,b) end end`. `os.remove` is a function and removes it afterwards.

os.time() returns fractional seconds, os.date() takes them, and os.clock() is the same wall clock

REFUTED in part. `os.time()` does carry a fraction — 0.811 of a second on one read — so `TTI_Apps/Pong.tspa:192-193` pacing an animation with `os.time() + 0.2` works. But `os.date("!!%H:%M:%S", t)` **accepts the float** and prints the same string as the floored value, so `mainframeInfo.tsp:33-36`'s comment, "os.date() only accepts an integer timestamp", does not hold on a DMM6500. And `os.clock()` is **not a process clock**: it returns the same epoch value as `os.time()`, the two differing by 10 μ s on one read. A difference across `delay(0.5)` still measures 0.50008 s, so `Speed_Scanning.tsp:9,59` bracketing a scan with it is sound — but the absolute value is wall time, not time since start. No manual has an *entry* for `os.time`, `os.date` or `os.clock`, but the reference manual uses both in examples — `os.time({year=...})` and `os.date('%c', gettime())` — so do not read this as absent, only as unspecified.

- probe: `do local a=os.time() local b=os.clock() local ok,d=pcall(os.date,"!!%H:%M:%S",a) print("OSTIME|t=..toString(a)..|frac=..toString(a-math.floor(a))..|clock=..toString(b)..|dateok=..toString(ok)..|date=..toString(d)) end` — pass the raw float, not the floored one, or the claim's own question goes unasked.

Nine introspection symbols appear in no DMM6500 manual because the DMM6500 does not have them

REFUTED. `Instrument_Examples/MP5000/mainframeInfo.tsp:46-58` prints `localnode.manufacturer`, `localnode.revision`, `localnode.description`, `localnode.autolinefreq`, `slot.autostart`, `gettimezone()`, `table.getn(userstring.table())`, `table.getn(script.table())` and `eventlog.count`. Sibling product, and **every one of them is nil here** — `script.table` and `userstring.table` are not functions but nil fields, so calling either raises "attempt to call field 'table' (a nil value)", and `gettimezone()` raises "attempt to call a nil value". Only `slot[1]` is a table. The manuals are silent because there is nothing to document: that script runs on an MP5000, not on this box.

- probe:


```
print("INTRO|mfr=..toString(localnode.manufacturer)..|rev=..toString(localnode.revision)..|desc=..toString(localnode.description)..|alf=..toString(localnode.autolinefreq)..|evc=..toString(eventlog.count)..|gtz=..type(gettimezone)..|st=..type(script.table)..|ut=..type(userstring.table)..|sas=..toString(slot[1]))
```

 — `type()` first, then a guarded call, because a nil field and a function that fails read the same in prose and not in a probe.

Keithley's own host code assumes five things the manual never states

MEASURED, for the three the instrument can answer. The reload collision is real: a second `loadscript` on a live name files **1408 "A script with the same name already exists"**, and `script.delete` itself logs **-104 "Data type error"** whichever way the script is named. The acknowledgement assumption holds too — `localnode.prompts` reads `localnode.DISABLE`, so the socket returns nothing until a script prints.

Assumption	Evidence
Delete the script name before sending: <code>if loadfuncs ~= nil then script.delete('loadfuncs')</code> end . Twenty-six host files in Python, C#, VB and Go open with this line.	Instrument_Examples/General/Send_a_Script_File_to_A_Keithley_Touchscreen_Instrument/...Sockets_Python.py:225
The 1024-byte ceiling is per line, not per write. The whole script goes in one <code>send</code> .	same file, :228
Nagle must be off: <code>setsockopt(IPPROTO_TCP, TCP_NODELAY, 1)</code> .	same file, :51
Nothing in the protocol acknowledges a command, so the acknowledgement is built out of <code>print</code> . Three of the four host-called functions end <code>print("ok")</code> ; <code>get_data</code> answers with the data itself, and the file ends <code>print("functions loaded")</code> so the host can see the load finish. The host blocks on <code>recv</code> .	00_Single_Channel/functions_V3.lua:39,55,61 ; Stream_TTI_SMU.py:102,108,147
A query needs 100 ms between the write and the read.	Drivers/DMM6500_DAO6510/DMM6500_Python_Sockets_Driver/DMM6500_Sockets_Driver.py:100

- `probe: send loadscript ktst3 / print("a") / endscript TWICE with no delete between, then drain the log with eventlog.next(eventlog.SEV_ALL); clean up with do if ktst3~=nil then pcall(function() script.delete(ktst3) end) ktst3=nil end end and drain the -104 that leaves behind. print("HOST|prompts="..tostring(localnode.prompts).."|showevents="..tostring(localnode.showevents)) settles the acknowledgement.`
- `no probe: TCP_NODELAY` and the 100 ms sleep are host socket settings with no instrument readback. The per-line 1024-byte ceiling is assumed throughout this file rather than tested: every probe here is kept under it because past it the instrument answers -363 with no sentinel.

Two Keithley scripts tell you to replace `waitcomplete()` with SRQ or *OPC when a host is driving

UNTESTABLE HERE — the stall needs a live overlapped operation, and the only one this bench can make is a trigger-model initiate, which a host command aborts before it can be waited on. **This is vendor practice, not a measurement:** `Application_Specific/Battery_Test/DC-IR/Battery_DCIR.tspa:536` reads "or wait SRQ or *OPC if in python" and

`Application_Specific/Rds(On)_of_SiC_MOSFET/Rdson_digitize_with_timing_marks.tsp:286` says "comment this out and wait SRQ if calling from Python" — an imperative. Neither says it stalls anything, and both are SMU-paired application scripts. The manual says only that the call "suspends the execution of commands", and nothing about the remote interface. The idle cost is settled and small: with nothing overlapped `waitcomplete()` returns in **15 µs** and `waitcomplete(0)` in **9 µs**, so the call is free when there is nothing to wait for and any stall belongs to a real overlapped operation rather than to the call.

- `probe: do local t0=os.clock() waitcomplete() local t1=os.clock() local t2=os.clock() waitcomplete(0) local t3=os.clock() print("WC|idle="..tostring(t1-t0).."|grp0="..tostring(t3-t2)) end`
- `no probe: the stall itself needs a live overlapped operation on another interface. A trigger-model initiate of our own cannot stand in – a host command aborts an initiated model, measured below, so the overlap ends before it can be waited on.`

The trigger model

`trigger.model.state()` reports a lag, not a readiness — and it has nothing to do with the buffer

REFUTED, and this entry used to say the opposite. The time for an armed `LoopUntilEvent` model to report `trigger.STATE_WAITING` is **flat at about 0.19 s from 21,100 to 2,800,000 readings** — least-squares slope $+1.6\text{e-}9 \pm 5.7\text{e-}9$ s per reading, $t = 0.28$, which is 4.5 ms across a 132× range. `digitize.count`, `samplerate` over a 1000× range, `template position`, a host `buf.clear()` and the poll cadence are all nulls. The earlier "about 1 ms at 21,100 readings" was never measured: thirteen arms at that size give **0.1668 to 0.2154 s**. What does move it is the **prior acquisition** — 0.228 s after aborting a model that was itself waiting, 0.191 s after `load("Empty")`, 0.177 s after a `SimpleLoop` run to idle, 0.164 s after a completed `digitize.read()`.

The wait block is live about 2 ms after `initiate()` ; the state call just does not admit it for another 190. Two independent derivations agree: `pre-roll start = fire_s - N5/fs - n_pre/fs` gives 1.6–2.6 ms over six captures, and the block trajectory has `BUFFER_CLEAR` done at 0.0003 s and the zero `DELAY_CONSTANT` at 0.0012 s. Across twelve latency captures `STATE_WAITING` was **never once observed**, because the signal fired inside one ramp period of the wait going live. So the state call is a lagging indicator and arming is not what costs the 190 ms. No vendor example loads `LoopUntilEvent`, and nothing in the tree or the manual states a block latency; `docs/TRIGGER.md:96` records that loading it destroys blocks 7–8, which bounds what "armed against a buffer" can mean.

- probe: initiate a `LoopUntilEvent` model, poll `trigger.model.state()` from inside the instrument, and time the transition at two buffer sizes – but give the loop a delay. A spin that never yields starves the trigger engine: 56,102 state calls over 3 s and the model never left block 2, while 10, 1 and 0.2 ms cadences are indistinguishable. Full working in `/tmp/latency.md` and `/tmp/settle.md` .

A measuring model does NOT survive commands on the remote interface, and nor does a waiting one

REFUTED. Three host scripts query a meter in a loop for the whole length of an initiated model, and each depends on the model continuing – `get_data()` blocks inside the instrument until the buffer grows, so an abort on the first query would deadlock the host.

`Streaming_Examples/00_Single_Channel/Stream_DMM6500.py:74,86` streams chunks;

`01_Dual_Meter/KEIDMM6500_Stream_Measured_Dual_Meter.py:335,351` runs for 11 hours;

`DAQ6510_Long_Term_Scan_with_Plotting_TSP.py:264,276` prints `defbuffer1.n` in a `while True` . One of the three ends with an explicit `trigger.model.abort()` ; the other two let the host exit and leave the model running. **Only two are on a raw socket** (`Stream_DMM6500.py:105–106` and `DAQ6510_Long_Term...py:233` , both port 5025) and only one of those is a DMM6500 – the dual-meter script opens `USB0::0x05E6::0x6510::...:INSTR` under PyVISA, so it is USBTMC and one of its two boxes is a DAQ6510. One `print(defbuffer1.n)` from a socket aborts both shapes:

model	first query	second query, 1 s later	log
inside <code>BLOCK_MEASURE_DIGITIZE</code> , 20,000 readings at 1 kS/s	<code>STATE_RUNNING</code> , n = 2	<code>STATE_ABORTED</code> , n = 140	2731 , 2728
parked in <code>BLOCK_WAIT</code> , <code>EVENT_ANALOGTRIGGER</code> that cannot fire	<code>STATE_RUNNING</code> , n = 12	<code>STATE_ABORTED</code> , n = 200	2731 , 2728

The block type is not the variable. Both models above were digitizing when the abort landed, and `docs/TRIGGER.md:156` shows the same thing from the other direction – readings accumulating from 4,500 to 14,159 as each abort arrived. So "executing a measure block, never parked in a wait" bounded the wrong thing, and this entry used to say so. The first query is still answered truthfully, which is what makes it expensive to notice: the abort lands behind the reply and the model runs on about 140 ms. **Two candidate variables are still open** and worth one probe each – `defbuffer1` against a buffer made for the purpose, and a `printbuffer` read against a bare state poll. The residual conflict with the vendor is `Stream_DMM6500.py` alone, the one cited script that is both a DMM6500 and on a socket. The manual's "Aborting the trigger model" gives three stops and no command-interface effect.

- probe: two steps, because the second command is the hypothesis. (a) one statement that arms and returns – `do dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=1000 defbuffer1.clear() eventlog.clear() trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,20000) trigger.model.initiate() print("A_ARM|st="..tostring(trigger.model.state())) end.` (b) from the host, `print("A_POLL|n="..tostring(defbuffer1.n)..|st="..tostring(trigger.model.state()))` three times a second apart, then drain. For the waiting case put a 200–reading measure block first – an `EVENT_ANALOGTRIGGER` wait cannot be block 1, the comparator only evaluates against a live stream – and set `edge.level` out of reach of the input, which a 2000–sample read measures rather than assumes (± 10 mV idle here, so 9.9 V on the 10 V range).

Read a digitize block back: the constant is rewritten, and a template fixes both your count and your notify number

MEASURED. `trigger.BLOCK_DIGITIZE` exists and a block set with it **reads back as** `MEASURE_DIGITIZE` , confirming the rewrite.

`trigger.COUNT_AUTO` is 2147483647 and `COUNT_INFINITE` 2147483646; a hand-built `BLOCK_MEASURE_DIGITIZE` passed `COUNT_AUTO` took exactly the 7 readings `dmm.digitize.count` held. The notify collision is real too: a `LoopUntilEvent` template expands to six blocks ending 6) `NOTIFY NOTIFY: 2`, and `trigger.extout.stimulus` already defaults to `trigger.EVENT_NOTIFY2` – so a stimulus of ours on `NOTIFY2` collides with a template's end-of-capture. `BLOCK_DIGITIZE` occurs zero times in the manual, and four examples set a block with it, two of them DMM6500: `Streaming_Examples/00_Single_Channel/functions_V3.lua:36` , `.../functions_V4_EXT_Trigger.lua:36` , `Amp-Hour_Measurement/Amp-Hours.tsp:61,63` . `Amp-Hours.tsp:61` digitizes without end by passing `trigger.COUNT_INFINITE` ; :63 is `COUNT_STOP` , commented "stop making digitized measurements". The manual admits one legacy spelling, `BLOCK_MEASURE` , and says the instrument rewrites it to `BLOCK_MEASURE_DIGITIZE` – so read a block back before trusting a name. The **count** is not undocumented: a measure or digitize block with no count gets 1, stated with `setblock(1, trigger.BLOCK_MEASURE)` as the worked example, so only the spelling belongs here. For the count, `LIV/completePulse_CaseI_2601B_PULSE_DMM.tsp:288` and `LIV/measurementLIV_CaseI_2601B_PULSE_DMM.tsp:232` pass `trigger.COUNT_AUTO` after setting `dmm.digitize.count` ; the manual gives that constant one phrase, "Use most recent count value", and never says which count it means. A template bakes a literal, so only a hand-built block reads the setting. The same split shows in the event numbering: `Grading_and_Binning_Resistors/Grading_and_Binning_Resistors.tsp:50` routes a `GradeBinning` template's end-of-test with `trigger.digout[6].stimulus = trigger.EVENT_NOTIFY2` , commented "Trigger pulse is output when the Notify Block generates an event" – the same notify number the measured `LoopUntilEvent` expansion ends on, so a stimulus or a block of ours on `EVENT_NOTIFY2` collides with a template's end-of-capture. The `GradeBinning` page says `EXTERNAL TRIGGER OUT` is asserted at the end of each measurement and names no event.

- probe: `print(tostring(trigger.BLOCK_DIGITIZE)) dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_DIGITIZE,defbuffer1,5) print(trigger.model.getblocklist()) print(eventlog.next(eventlog.SEV_ALL))` – a nil constant refutes it; a block list naming `MEASURE_DIGITIZE` confirms the rewrite.

- probe: `dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.count=7 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1) trigger.model.setblock(2,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,trigger.COUNT_AUTO) trigger.model.initiate() waitcomplete() print(defbuffer1.n, dmm.digitize.count) print(eventlog.next(eventlog.SEV_ALL))` — seven readings confirms it; no signal needed.
- probe: `trigger.model.load("Empty") trigger.model.load("LoopUntilEvent", trigger.EVENT_EXTERNAL, 50) print(trigger.model.getblocklist()) print(eventlog.next(eventlog.SEV_ALL)) trigger.model.load("Empty")` — read the notify number off the last block. No card and no signal, since LoopUntilEvent drives no digital line.

A branch to block 0 ends the model

MEASURED. `LIV/completePulse_CaseI_2601B_PULSE_DMM.tsp:291` closes a model with `BLOCK_BRANCH_ALWAYS, 0` after a counter loop, and three sibling scripts do the same, five times over — all `Application_Specific/LIV_Diode_Testing/` scripts pairing a 2601B-PULSE with a "DMM7510 or DMM6510", building the blocks on `node[2]`, the DMM. The manual defines `branchToBlock` as "The block number to execute", gives 0 no meaning, and its own example branches from block 6 to block 20 in a six-block model. The instrument takes it: the block reads back as `3) BRANCH_ALWAYS BRANCH_BLOCK: 0`, the model runs, stores its **one** reading and finishes `STATE_IDLE` with nothing worse than `2731` and `2732` in the log. **0 terminates.**

- probe: `do eventlog.clear() dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=1000 dmm.digitize.count=1 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1) trigger.model.setblock(2,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,1) local ok=pcall(trigger.model.setblock,3,trigger.BLOCK_BRANCH_ALWAYS,0) print("ZERO|set="..tostring(ok)) end — then getblocklist(), because a setblock that was refused returns true anyway, and only then initiate and read defbuffer1.n with trigger.model.state().`

A wait on a line event arms with nothing acquiring, and one BNC releases it

MEASURED for the wait; the cable half is untested. A `BLOCK_WAIT, trigger.EVENT_EXTERNAL` as block 2, with the measure block behind it and nothing on the terminals, parks at `STATE_WAITING, block 2, n = 0` — so a line-event wait does arm with nothing acquiring, and the analog trigger's need for a live sample stream is the comparator's restriction and not the wait block's. `trigger.extin` and `trigger.extout` are both tables, `trigger.extout.assert` is a function, and with no card fitted `trigger.extout.stimulus` reads `trigger.EVENT_NOTIFY2` and `trigger.extin.edge` reads `trigger.EDGE_FALLING`. `LIV/completePulse_CaseI_2601B_PULSE_DMM.tsp:286` arms the meter on a hardware edge before any acquisition: block 1 clears the buffer, block 2 is `BLOCK_WAIT, EVENT_TSPLINK1, CLEAR_NEVER`, and the measure block is block 4. The omission is the point — the `BLOCK_WAIT` event table lists `EVENT_ANALOGTRIGGER` beside the line events with no distinction, so nothing marks the analog trigger as the one event that needs a live sample stream. That restriction belongs to the comparator, not to wait blocks. The same wait releases from the same box: `00_Single_Channel/functions_V4_EXT_Trigger.lua:14–15,30–37` sets `trigger.extout.stimulus = trigger.EVENT_NOTIFY1` with a block-1 NOTIFY, so EXTERNAL TRIGGER OUT fires when block 1 executes — and a BNC from OUT to IN lets the instrument satisfy its own `EVENT_EXTERNAL` wait, with `trigger.extout.assert()` releasing it from a socket.

`00_Single_Channel/functions_V3.lua:14–15,30–37` is the same shape over TSP-Link line 1 in `MODE_TRIGGER_OPEN_DRAIN`, needing no cable but the KTTI-TSP card. The manual is silent on a node waiting on a line it drives, and `trigger.extin` and `trigger.extout` carry no accessory-card note while every `digio.*` and `trigger.digin / digout` page does. Both probes substitute `EVENT_EXTERNAL` for the script's `EVENT_TSPLINK1` deliberately: the claim is about line events as a class, and the external one is the representative that needs no accessory card.

- probe: `do eventlog.clear() dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=1000 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1) trigger.model.setblock(2,trigger.BLOCK_WAIT,trigger.EVENT_EXTERNAL) trigger.model.setblock(3,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,10) trigger.model.initiate() delay(0.5) local a,b,c=trigger.model.state() print("LINEWAIT|st="..tostring(a).."|blk="..tostring(c).."|n="..tostring(defbuffer1.n)) trigger.model.abort() end` — one statement, so no host command can abort it mid-wait.
- no probe: the loopback needs one BNC from EXTERNAL TRIGGER OUT to EXTERNAL TRIGGER IN, and plugging it in costs a human at the bench. The readbacks above are the no-cable control, and they already show the wait parks.

A backward BRANCH_ALWAYS builds a model that cannot go idle

MEASURED. A count-1 measure block with `BLOCK_BRANCH_ALWAYS, 2` behind it acquired **500 readings in the first half-second and 1000 by the next**, held `STATE_RUNNING` throughout, and reached `STATE_ABORTED` only when `abort()` arrived — never `STATE_IDLE`. At 1 kS/s that is the full sample rate, so the loop costs nothing per lap. `01_Dual_Meter/functions_2021-03-06.lua:65` closes with `BLOCK_BRANCH_ALWAYS, 5` around a count-1 measure block. The model never reaches an end, `waitcomplete()` would never return, and the host's `abort()` is the only exit. `GAS/MS01_SetupAndRun.tsp:60,62` shows the escape its author abandoned, commented out: `BLOCK_BRANCH_ON_EVENT, trigger.EVENT_DISPLAY, 8` with `setblock(8, BLOCK_NOP)`, labelled "if the trigger key is pressed branch to where the trigger model is stopped".

- probe: `dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE defbuffer1.capacity=10000 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1) trigger.model.setblock(2,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,1) trigger.model.setblock(3,trigger.BLOCK_BRANCH_ALWAYS,2) trigger.model.initiate() delay(0.5) print(defbuffer1.n) print(trigger.model.state()) trigger.model.abort() delay(0.2) print(defbuffer1.n) print(trigger.model.state())` — n rising then frozen, and never IDLE until the abort. Do not call `waitcomplete()` on this model.
- no probe: the abandoned TRIGGER-key escape needs a finger on the key, so `EVENT_DISPLAY` cannot be synthesised over a socket

Poll trigger.model.state() instead of blocking on waitcomplete()

MEASURED. One chunk, never from the host: mid-run the state is `STATE_WAITING` at block 2; `abort()` then `waitcomplete()` gives `STATE_ABORTED` at block 2; a re-`initiate()` returns to `STATE_WAITING` at block 2 and ran the measure block again, `defbuffer1.n` reaching 10 for two passes of a count-5 block. So **WAITING is a mid-run state, the abort/waitcomplete/re-initiate sequence works**, and the log carries 2731 / 2728 once per pass and nothing else. `TTI_Apps/KE_DAQ6510_Demo.tsp:35` is the clean form, while `present_state == trigger.STATE_WAITING` or `present_state == trigger.STATE_RUNNING` do — WAITING is a mid-run state, not a stall. `TTI_Apps/Resistance_Tolerance_Meter.tspa:243,247` takes all three returns and uses `block_num > 9` as a progress counter driven by the 0.25 s `display.OBJ_TIMER` at :346 ; at :208-211 it re-arms with `abort()`, `waitcomplete()`, `initiate()`, though at :137 and :252 it aborts bare. Every trigger-model example in the reference manual blocks on `waitcomplete()`, and no text pairs `abort()` with `waitcomplete()` before a re-initiate.

- probe: one chunk, never from the host — `dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE` `trigger.model.load("Empty")`
`trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1)`
`trigger.model.setblock(2,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,5)`
`trigger.model.setblock(3,trigger.BLOCK_WAIT,trigger.EVENT_EXTERNAL)` `trigger.model.initiate()` `delay(0.4)` local
`a,b,c=trigger.model.state()` `print(a,b,c)` `trigger.model.abort()` `waitcomplete()` local `d,e,f=trigger.model.state()` `print(d,e,f)`
`trigger.model.initiate()` `delay(0.2)` `print(trigger.model.state())` `trigger.model.abort()`

A timer count is events per start, not events per model, and a timer can be aliased

MEASURED. Timer 2 with `count = 1`, started by `EVENT_TIMER1`, waited on once per lap of a three-lap `BLOCK_BRANCH_COUNTER`, produced **three readings** — so the count is per start, not per model. The alias works and so does writing through it while enabled: local `t = trigger.timer[1]` then `t.enable=1` `t.delay=0.25` `t.count=2` reads back as 0.25, 2 and `trigger.ON` on `trigger.timer[1]` itself, with **nothing filed**. `Rds(On)_of_SiC_MOSFET/Rdson_digitize_with_timing_marks.tsp:110-111,126-127` chains timers and leaves the downstream counts at 1: timer 1 takes `count = NumPulses`, timer 2 takes `measDelay.count = 1` with `start.stimulus = trigger.EVENT_TIMER1`. The model waits on timer 2 once per lap of a `BLOCK_BRANCH_COUNTER` loop, so a count of 1 fires `NumPulses` times. The manual says to "make sure the count value is the same or more than any count values expected in the trigger model" and supplies the correct reading elsewhere: the count "sets the number of events to generate each time the timer generates a trigger event". The same script names its timers — `pulsePeriod = trigger.timer[1]` at :90 — writes every setting through the alias, and sets `enable = 1` before `delay` and `count`, against the manual's advice to disable first.

- probe: `trigger.timer[1].reset()` `trigger.timer[2].reset()` `trigger.timer[1].delay=0.05` `trigger.timer[1].count=3`
`trigger.timer[1].start.stimulus=trigger.EVENT_NOTIFY1` `trigger.timer[1].start.generate=trigger.OFF` `trigger.timer[1].enable=1`
`trigger.timer[2].delay=0.01` `trigger.timer[2].count=1` `trigger.timer[2].start.stimulus=trigger.EVENT_TIMER1`
`trigger.timer[2].start.generate=trigger.OFF` `trigger.timer[2].enable=1` `trigger.model.load("Empty")`
`trigger.model.setblock(1,trigger.BLOCK_BUFFER_CLEAR,defbuffer1)`
`trigger.model.setblock(2,trigger.BLOCK_NOTIFY,trigger.EVENT_NOTIFY1)`
`trigger.model.setblock(3,trigger.BLOCK_WAIT,trigger.EVENT_TIMER2)`
`trigger.model.setblock(4,trigger.BLOCK_MEASURE_DIGITIZE,defbuffer1,1)`
`trigger.model.setblock(5,trigger.BLOCK_BRANCH_COUNTER,3,3)` `trigger.model.initiate()` `delay(2)` `print(defbuffer1.n,`
`trigger.model.state())` `trigger.model.abort()` `print(eventlog.next(eventlog.SEV_ALL))` — 896 bytes, so send it alone. Three readings from a count of 1 is the claim.
- probe: local `t=trigger.timer[1]` `t.enable=1` `t.delay=0.25` `t.count=2` `t.start.generate=trigger.OFF`
`print(trigger.timer[1].delay, trigger.timer[1].count, trigger.timer[1].enable, trigger.timer[1].start.generate)`
`print(eventlog.next(eventlog.SEV_ALL))` — 0.25, 2 and enabled proves both the alias and the write-while-enabled.

available(digio) works, and the manual's token list omits it

MEASURED. `available` is a function and **available(digio) is accepted**, returning `false` with nothing filed — so the token works and the manual's four-token list is incomplete. `digio`, `tsplink`, `channel`, `gplib` and `serial` are all tables here, so a nil namespace cannot be mistaken for a missing card: of the five only `available(channel)` is true, the other four false. `TTI_Apps/DIOControlFull.tspa:371` gates its whole digital-I/O setup on `if available(digio) == true` then. The `available()` entry lists four tokens — `channel`, `gplib`, `serial`, `tsplink` — while the Details on the same page name digital I/O among the accessory-card options this function exists "to check for".

- probe: `do print("AVAIL|digio="..type(digio)..)|tsplink="..type(tsplink)..)|channel="..type(channel))` local function `t(x)`
local `ok,v=pcall(available,x)` if `ok` then `return tostring(v)` end `return "ERR"` end
`print("AVAIL2|digio="..t(digio)..)|channel="..t(channel)..)|ev="..tostring(eventlog.getcount()))` end — read `type()` first, because on a box with no card a nil token and a false answer are different findings, and drain afterwards: a refused token files an event rather than raising.

The digitizer and reading buffers

The analog trigger level reads back changed, and it tracks the measured idle

MEASURED. On a 10 V digitize range, `dmm.digitize.analogtrigger.edge.level` written as 6.00 V reads back 5.99 V. Two runs apart the readback moved 20 mV, following the measured idle level. The level is adopted, not refused, at the top of the range. `dmm.digitize.range` on this app is 10, and the level range follows the active measurement range as documented. No example in the tree reads an `analogtrigger` attribute back, and the manual is silent on readback fidelity.

- `probe: write dmm.digitize.analogtrigger.edge.level = 6.0` on a 10 V range, read it back, change the applied idle level, read it back again

printbuffer takes a range the manual forbids

MEASURED. `printbuffer(1, defbuffer1.n, defbuffer1.readings)` over a five-reading buffer printed **exactly five real values and filed nothing. The manual forbids its own prescribed idiom.** Its `bufferVar.n` entry prescribes `printbuffer(1, bufferVar.n, bufferVar.readings)` as the way "to print all the readings in a buffer", and that form appears five times in the manual — while `printbuffer`'s own parameter table bounds `startIndex` by "more than one and less than endIndex" and `endIndex` by "less than the index of the last entry in the tables". Both bounds the table forbids are accepted, so the prescription is right and the table describes no behaviour this instrument has.

- `probe: do dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=1000 dmm.digitize.count=5 defbuffer1.clear() eventlog.clear() dmm.digitize.read() print("PB|n="..tostring(defbuffer1.n).."|ev="..tostring(eventlog.getcount())) end then printbuffer(1,defbuffer1.n,defbuffer1.readings)`, then drain. The clear matters: without it `defbuffer1` still holds the previous probe's readings and five values prove nothing.

The LAN interface, not the digitizer, sets the streaming ceiling

MEASURED, for the response size and the digitizer's own ceiling. A `printbuffer(1,249,defbuffer1.readings)` under `format.REAL32` came back as **exactly 999 bytes in one chunk**, so the vendor's `chunkSize = 249` really is one Ethernet frame's worth and the arithmetic in this entry is right. The digitizer stops at **1,000,000 S/s**: asking for 2,000,000 or 10,000,000 leaves `dmm.digitize.samplerate` reading 1000000 and files `1130` "Parameter digitize sample rate, expected value from 1000 to 1000000" — the request is refused, not clamped silently, and `format.SREAL` exists beside `REAL`, `REAL32`, `REAL64` and `DREAL`. `Streaming_Examples/00_Single_Channel/Stream_DMM6500.py:16` ships `sample_rate = 50000` and annotates it: "60kS/s is the max rate we have observed under certain conditions/circumstances. To attain higher sampling and data transfer rates, use USB." The same file caps a poll at `chunkSize = 249` `format.REAL32` readings — a `#0` header, four bytes a reading and a newline is 999 bytes, inside one Ethernet frame. 249 is tuning, not a ceiling: `01_Dual_Meter/KEIDMM6500_Stream_Measured_Dual_Meter.py:314,351` moves 1000 `REAL32` readings, about 4 kB, in one response and queries a second instrument for another — and being USBTMC it is weak evidence about a LAN ceiling. TCP segmentation hides the frame size, so the frame cannot be the cause. The manual ranks the interfaces — "USB has the fastest transfer rate" — and gives no figure and no response size.

- `probe: host counts the response bytes. dmm.digitize.count=300 dmm.digitize.read() format.data=format.REAL32 printbuffer(1,249,defbuffer1.readings)` then restore `format.data=format.ASCII` — expect exactly 999 bytes. For the two-column form, `dmm.digitize.count=1000 dmm.digitize.read() printbuffer(1,1000,defbuffer1.readings,defbuffer1.relativetimestamps)` — expect 8003 bytes and no truncation.
- no probe: the sustained 60 kS/s figure needs a host-side timed extraction loop; from inside the instrument `printbuffer` blocks on socket backpressure, which bounds the rate but does not measure it

buffer.getstats(buf).n keeps counting after a buffer laps

MEASURED. A 100-reading `FILL_CONTINUOUS` buffer filled by a 500-reading measure block: `buf.n` pins at **100**, `startindex` 1, `endindex` 100 — and `buffer.getstats(buf).n` reads **500**. A second 500 into the same buffer takes it to **1000**, so it is cumulative since the buffer was made rather than per acquisition. It is also monotonic *during* a capture, which is the only property a poll loop needs: sampled a second apart inside one 30,000-reading block it read **10001, 20004, 30000** while `endindex` wrapped from 4 to 7. `buf.clear()` resets it to 0. So it is a strictly better progress signal than `endindex` for a lapping buffer, and the one thing to know is that `dmm.digitize.read()` asking for more readings than the destination's capacity stores **nothing at all** — `n = 0`, two 4928 entries — so the count has to come from a measure block, not a bare read. `.n` counts every reading ever stored, overwritten ones included, so it is the progress counter a lapping buffer still has. **The vendor leans on it across eight laps.** `01_Dual_Meter/functions_2021-03-06.lua:48` sets `samplebuff.fillmode = buffer.FILL_CONTINUOUS`, and its host runs **11 hours at 1 kS/s into a 5,000,000-reading buffer — about 39.6 M readings, eight times round** — polling `getstats().n` at :94 throughout as the wait condition for the next chunk. Three more do the same with a private tally beside it: `00_Single_Channel/functions_V3.lua:69`, the line verbatim at `DMM7510/Stream_Digitized_Measurements/Sockets/Single_Box/functions_V3.lua:56`, and `Amp-Hours.tsp:81`. The first polls every 0.5 ms, so the call is cheap at size — though `EC_Framework.tsp:156-182` memoises `buffer.getstats` behind a hit-and-miss cache, so the vendor's own practice disagrees with itself on the cost. The manual gives `n` one line, "The number of data points on which the statistics are based".

- `probe: do KB=buffer.make(100,buffer.STYLE_STANDARD) KB.fillmode=1 eventlog.clear() dmm.digitize.func=dmm.FUNC_DIGITIZE_VOLTAGE dmm.digitize.range=10 dmm.digitize.samplerate=10000 trigger.model.load("Empty") trigger.model.setblock(1,trigger.BLOCK_MEASURE_DIGITIZE,KB,500) trigger.model.initiate() waitcomplete() local s=buffer.getstats(KB) print("GS1|n="..tostring(KB.n).."|si="..tostring(KB.startindex).."|ei="..tostring(KB.endindex).."|gsn="..tostring(s.n)) end — fillmode = 1, numerically: buffer.FILL_CONTINUOUS does not exist on this firmware. Delete KB in a pcall and only nil it if the delete took.`

The real ceiling is 7,080,255 readings, across all buffers together

MEASURED, and the firmware states it itself. `buffer.make(9000000, buffer.STYLE_STANDARD)` files `4920 "Invalid size: Reading buffer capacity must be between 10 and 7080255 readings"` — so neither published figure is right, and the feature list's "seven million" is the closer of the two. The limit is on the **total**: with this app's 1,437,294-reading buffer and both default buffers at 100,000, a 6,000,000-reading request files `4919 "Available capacity for all reading buffers exceeded, remaining capacity is 5419050 readings"`, and 1,437,294 + 200,000 + 5,419,050 comes to 7,056,344. **A refused buffer.make returns nil while pcall reports success** — the only verdict is the readback, and deleting what it returned files `1138 "Parameter 1, expected type 'reading buffer', but found type 'unknown'"`. `bufferVar.capacity` says the

overall capacity "can be up to 6,000,000 readings for standard reading buffers"; the feature list says "up to seven million". One example takes the feature list at its word and lands on the right number:

```
Use_the_DAO6510_to_Control_a_Temperature_Chamber/F4T_EnvController_DAO6510.tsp:521 sets defbuffer1.capacity = 7000000.
GAS/MS01_SetupAndRun.tsp:121 sidesteps both — asked for 10,000,000 by its own call site at :212, it guards on if buffersize > 7000000
and sets capacity = 0 to maximise. Capacity, not count, is the limit on a capture: Amp-Hours.tsp:48 reads "Changing count is optional. The
reading buffer capacity is the determining factor", beside dmm.digitize.count = 1 -- CANNOT be zero; 1 to 55Million.
```

- probe: ask for MORE than the box can ever grant and read 4920's own text, which costs no allocation — do eventlog.clear()
local ok,b=pcall(buffer.make,9000000,buffer.STYLE_STANDARD) if ok and type(b)=="table" then pcall(buffer.delete,b) end local
e,m=eventlog.next(eventlog.SEV_ALL) print("P00L|ok="..tostring(ok).."|bt="..type(b).."|e="..tostring(e).."|m="..tostring(m))
end . Do not walk upward through real sizes: a 4,500,000-reading buffer.make did not return inside 90 s and ended in -286, which looks
exactly like a hung instrument.

buffer.make on a name that already exists: the manual says overwrite

MEASURED. The second buffer.make on the same global goes through silently: capacity 20 then capacity 30, nothing filed. After one
buffer.delete a fresh 20 can be made again, and the pool reads back unchanged afterwards — 100,000 on each default buffer and 1,437,294 on
this app's — so the overwritten buffer is not stranded. The manual documents the same-name case as "the existing buffer is overwritten by the new
buffer", and TSP-side setup in the tree re-runs buffer.make() on the same global without deleting. The tree holds one buffer.delete() in total,
at teardown of the 5,000,000-reading buffer in KEIDMM6500_Stream_Measured_Dual_Meter.py:359.

- probe: do eventlog.clear() KTHA=buffer.make(20,buffer.STYLE_STANDARD) local c1=KTHA.capacity
KTHA=buffer.make(30,buffer.STYLE_STANDARD)
print("SAMENAME|cap1="..tostring(c1).."|cap2="..tostring(KTHA.capacity).."|ev="..tostring(eventlog.getcount())) end — a
distinctive global, because buffer.make might have refused a name already bound. Whether the first 20 came back is the 4919 readback
above, taken before and after.

Indexing a buffer object returns the reading, writable styles included

MEASURED. Both forms agree, on both styles. A STYLE_WRITABLE buffer written with 1.25 reads b[1] 1.25 and b.readings[1] 1.25, units Volt
DC, nothing filed; on the digitizer-filled defbuffer1, defbuffer1[1] and defbuffer1.readings[1] both read -0.0092203052674 and
defbuffer1[defbuffer1.n] gives the last reading. TTI_Apps/Probe_Hold.tspa:227 indexes a STYLE_WRITABLE buffer as
App_buffer[App_buffer.n], :275 reads defbuffer1[endindex], and Battery_DCIR.tspa:597 reads dcirBuffer[j]. The documented form
is bufferVar.readings[N], and buffer.make() warns that not all remote commands are "compatible" with the writable styles.

- probe: do eventlog.clear() KB=buffer.make(10,buffer.STYLE_WRITABLE)
buffer.write.format(KB,buffer.UNIT_VOLT,buffer.DIGITS_6_5) buffer.write.reading(KB,1.25)
print("WIDX|idx="..tostring(KB[1]).."|sub="..tostring(KB.readings[1]).."|n="..tostring(KB.n).."|unit="..tostring(KB.units[1])
) end, then the same two forms on defbuffer1. Delete KB in a pcall, once.